St. Francis Institute of Technology
(An Autonomous Institution)
AICTE Approved | Affiliated to University of Mumbai

A+ Grade by NAAC: CMPN, EXTC, INFT NBA Accredited: ISO 9001:2015 Certified

Department of Information Technology

A.Y. 2025-2026
Class: BE-IT A/B, Semester: VII
Subject: Secure Application Development Lab

Student Name: Tanmay Bhatkar Student Roll No: 14

Experiment – 2 : Study of Secure Software Development Life Cycle (SDLC)

Aim: To study secure Software Development Life Cycle (SDLC).

Objectives: After study of this experiment, the student will be able to

- Understand different steps of SDLC .
- Identify and learn different standards of cyber security.

Lab objective mapped: ITL703.2 : To **understand** the methodologies and standards for developing secure code

Prerequisite: Basic Knowledge of different stages SDLC

Requirements: Personal Computer, Windows operating system browser, Internet Connection etc.

Pre-Experiment Theory:

What is Secure SDLC and Why is it important ?

Secure System Development Life Cycle (SecSDLC) is defined as the set of procedures that are executed in a sequence in the Software Development Life Cycle (SDLC). It is designed such that it can help developers to create software and applications in a way that reduces the security risks at later stages significantly from the start. The Secure System Development Life Cycle (SecSDLC) is similar to Software Development Life Cycle (SDLC), but they differ in terms of the activities that are carried out in each phase of the cycle. SecSDLC eliminates security vulnerabilities. Its process involves identification of certain threats and the risks they impose on a system as well as the needed implementation of security controls to counter, remove and manage the risks involved. Whereas, in the SDLC process, the focus is mainly on the designs and implementations of an information system.

Phases involved in SecSDLC are:

- **System Investigation:** This process is started by the officials/directives working at the top-level management in the organization. The objectives and goals of the project are considered priory in order to execute this process. An Information Security Policy is defined which contains the descriptions of

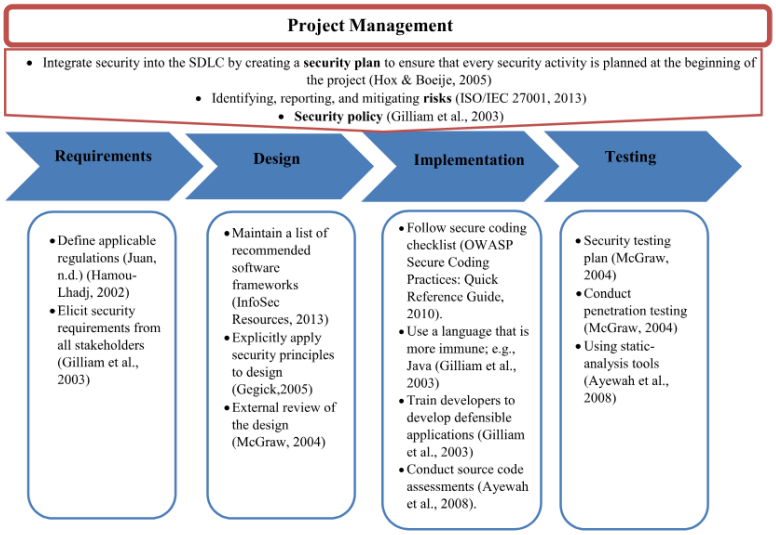
security applications and programs installed along with their implementations in the organization's system.

- **System Analysis:** In this phase, detailed document analysis of the documents from the System Investigation phase are done. Already existing security policies, applications and software are analyzed in order to check for different flaws and vulnerabilities in the system. Upcoming threat possibilities are also analyzed. Risk management comes under this process only.
- **Logical Design:** The Logical Design phase deals with the development of tools and following blueprints that are involved in various information security policies, their applications and software. Backup and recovery policies are also drafted in order to prevent future losses. In case of any disaster, the steps to take in business are also planned. The decision to outsource the company project is decided in this phase. It is analyzed whether the project can be completed in the company itself or it needs to be sent to another company for the specific task.
- **Physical Design:** The technical teams acquire the tools and blueprints needed for the implementation of the software and application of the system security. During this phase, different solutions are investigated for any unforeseen issues, which may be encountered in the future. They are analyzed and written down in order to cover most of the vulnerabilities that were missed during the analysis phase.
- **Implementation:** The solution decided in earlier phases is made final whether the project is in-house or outsourced. The proper documentation is provided of the product in order to meet the requirements specified for the project to be met. Implementation and integration process of the project are carried out with the help of various teams aggressively testing whether the product meets the system requirements specified in the system documentation.
- **Maintenance:** After the implementation of the security program, it must be ensured that it is functioning properly and is managed accordingly. The security program must be kept up to date accordingly in order to counter new threats that can be left unseen at the time of design.

Procedure:

- Study the case study from references and discuss the proposed model to integrate security in SDLC.

  The case study from the article titled **"A model for integrating security into software development life cycle"** (source: Wiley Online Library) proposes a **structured model to embed security throughout the Software Development Life Cycle (SDLC)**, ensuring that security is **not an afterthought** but an integral component from the outset.



Project Management Integration

At the foundation of the model is strong project management oversight. The authors propose creating a security plan at the beginning of the project to ensure that all security activities are defined, tracked, and executed throughout the SDLC. Project management is also responsible for identifying, reporting, and mitigating risks (aligned with ISO/IEC 27001:2013), as well as enforcing a formal security policy to guide development activities

Requirements Phase

In the requirements phase, the model stresses the importance of:

- Defining applicable regulations and compliance needs.
- Eliciting security requirements from all stakeholders.

Design Phase

During the design phase, the model recommends:

- Maintaining a list of trusted and secure software frameworks.
- Explicitly applying security principles such as least privilege and defense in depth.
- Conducting external reviews of the design to identify architectural vulnerabilities.

Implementation Phase

The implementation phase focuses on:

- Adhering to secure coding practices (e.g., OWASP guidelines).
- Using programming languages that inherently reduce security risks (e.g., Java).
- Training developers in secure software development.
- Conducting source code assessments using automated tools.

Testing Phase ensures that security flaws are detected and resolved before deployment.

In this final phase of the SDLC, the model includes:

- Developing and executing a security testing plan.
- Conducting penetration tests to simulate real-world attacks.
- Utilizing static analysis tools to detect vulnerabilities in the source code.

The proposed model is comprehensive and emphasizes a security-by-design approach. By integrating security practices at every SDLC stage and enforcing oversight through project management, the model aims to produce secure, reliable software.

- Compare Software Development Life Cycle (SDLC) and Secure SDLC.

Parameter	Traditional SDLC	Secure SDLC (SSDLC)
Focus	Deliver functional software efficiently.	Integrate security throughout, from planning to maintenance.
Security Consideration	Often limited to the final testing phase.	Embedded in every phase, enabling early risk identification.
Requirement Gathering	Addresses functional and business needs.	Includes stakeholder-defined security requirements and threat modeling.
Design Phase	Focuses on architecture and usability.	Applies secure design principles and proactive threat modeling.
Implementation Phase	General coding standards with a functional focus.	Enforces secure coding standards, developer training, and uses tools like SAST/SCA.
Testing	Functional test suites; security testing is often separate or limited.	Continuous security testing integrated into CI/CD (e.g., SAST, DAST, pen testing).
Deployment & Maintenance	Manual configuration and reactive patching.	Automated secure deployment, runtime monitoring, and incident response readiness.
Cost & Team Culture	Faster delivery, but higher cost if security issues are found later.	Slightly slower initially, but lower long-term cost and promotes a security-first mindset.

- Describe how secure coding can be incorporated into the software development process.

  Secure coding is the practice of designing code that adheres to established security best practices, helping safeguard against vulnerabilities such as exploits, credential leakage, and exposure of personally identifiable information (PII) (Check Point). It can be effectively integrated into the software development process through a combination of cultural, procedural, and technical measures.
  Shift-Left Security and Developer Responsibility
  Security enforcement should begin at the earliest stages of development. Code security must be integrated directly into the CI/CD pipeline, ensuring that vulnerabilities are addressed proactively rather than post-deployment. Developers, rather than separate security teams, are increasingly recognized as directly responsible for maintaining the security of the code they write.

  Cultivating a Security-Conscious Culture
  An organizational culture of security is essential. All stakeholders—including developers, operations personnel, and product managers—should be engaged in

promoting secure development. Designating security-focused developers, known as security champions, within development teams helps maintain and scale security awareness and implementation throughout the development process.

Secure Processes Across the SDLC
Security measures must be embedded across all phases of the software development lifecycle, from requirements and design through development, testing, deployment, and maintenance. This includes the adoption of secure coding standards, structured code reviews, and defined protocols for handling vulnerabilities.

Automated Security Testing
Automated security scanning is critical for maintaining speed and consistency. While manual code reviews are valuable, automation enables continuous security assessment, identifying both visible and hidden risks across code and infrastructure. Integrating these tools into the CI/CD workflow supports real-time vulnerability detection and remediation.

Tooling Integration with Developer Workflows
To encourage adoption, security tools must be easy to use and integrate seamlessly with existing development environments. Tools that align with current workflows ensure that developers can maintain productivity while actively addressing security concerns.

Post-Experiments Exercise:

Extended Theory: Nil

Post Experimental Exercise:

Questions:

- List the major types of coding errors and their root causes.
- Describe good software development practices and explain how they affect application security.

Conclusion:

- Write what was performed in the experiment.
- Write the significance of the topic studied in the experiment.

References:

- Case study: https://onlinelibrary.wiley.com/doi/epdf/10.1002/sec.1700
- https://snyk.io/learn/secure-sdlc/
- https://www.checkpoint.com/cyber-hub/cloud-security/what-is-secure-coding/
- https://snyk.io/learn/secure-coding-practices/